

Guia de Estudo Completo do Artigo

“A multi-objective ant colony system algorithm for virtual machine placement in cloud computing” — Gao et al. (2013)

Material de apoio — Seminário de Algoritmos Bioinspirados

Lucas Rivetti Ian Nunes

Junho de 2026

Como usar este guia. Ele explica, do zero e em ordem, tudo que importa no artigo: os *conceitos* (nuvem, virtualização, alocação de VMs, otimização multiobjetivo, colônia de formigas), a *formulação matemática* com o significado de **cada variável**, o *algoritmo VMPACS* passo a passo, e os *experimentos, métricas e resultados*. No fim há um **glossário de símbolos** e uma seção de **prováveis perguntas da banca**. Os boxes azuis trazem a “intuição” de cada ideia.

Sumário

1	Visão geral em um parágrafo	1
2	Conceitos de base: computação em nuvem	1
2.1	Os três modelos de nuvem	1
2.2	Virtualização, hypervisor e VM	1
2.3	Por que energia e desperdício importam	2
3	O problema central: alocação de VMs (VM placement)	2
3.1	Por que é difícil (NP-difícil)	2
3.2	Abordagens da literatura (citadas no artigo)	2
4	Otimização multiobjetivo (a teoria por trás)	2
4.1	Vocabulário formal	2
4.2	Dominância de Pareto	2
4.3	Fronteira de Pareto	3
5	Colônia de Formigas (ACO)	3
5.1	A metáfora biológica	3
5.2	Variantes e a versão multiobjetivo	3
6	Modelagem dos recursos e dos dois objetivos	3
6.1	Objetivo 2: desperdício de recursos W_j	3
6.2	Objetivo 1: consumo de energia P_j	4
7	Formulação matemática completa	4

8	O algoritmo VMPACS (passo a passo)	5
8.1	Feromônio τ_{ij} e seu valor inicial τ_0	5
8.2	Heurística η_{ij} (o “conhecimento do problema”)	5
8.3	Como a formiga escolhe (regra pseudoaleatória proporcional)	6
8.4	Atualização do feromônio (local e global)	6
8.5	Pseudocódigo resumido	6
9	Metodologia experimental do artigo	7
9.1	Geração das instâncias (com correlação CPU/memória)	7
9.2	Parâmetros	7
10	Métricas usadas no artigo	7
10.1	ONVG — quantas soluções a fronteira tem	7
10.2	Spacing (SP) — quão uniforme é o espaçamento	7
11	Resultados do artigo	7
12	Conclusão do artigo	8
13	Glossário de símbolos	8
14	Prováveis perguntas da banca (e respostas)	8

1 Visão geral em um parágrafo

O artigo ataca o problema de **onde colocar cada máquina virtual (VM)** num *data center* de nuvem. Colocar bem as VMs importa porque define quanta **energia** o data center gasta e o quanto de **recurso** (CPU e memória) é desperdiçado. Como esses dois objetivos competem, o problema é *multiobjetivo*: não existe uma única solução ótima, mas um conjunto de bons compromissos (a *fronteira de Pareto*). Os autores propõem o **VMPACS**, uma *colônia de formigas* adaptada para achar esse conjunto, e mostram que ele supera um algoritmo genético (MGGA) e dois métodos de objetivo único (FFD e SACO).

2 Conceitos de base: computação em nuvem

2.1 Os três modelos de nuvem

- **IaaS** (Infraestrutura como Serviço): você aluga máquinas/recursos (ex.: Amazon EC2). É o nível em que vive o problema do artigo.
- **PaaS** (Plataforma como Serviço): você aluga um ambiente pronto para rodar aplicações.
- **SaaS** (Software como Serviço): você usa um software pela internet (ex.: Gmail).

2.2 Virtualização, hypervisor e VM

Virtualização é a técnica que divide um servidor físico em vários ambientes isolados, as **máquinas virtuais (VMs)**. Um programa chamado **hypervisor** controla esse compartilhamento. Assim, várias aplicações rodam “lado a lado” no mesmo hardware, cada uma achando que tem um computador só para si. Isso permite *computação sob demanda*: aloca-se CPU, memória e disco conforme a necessidade, em vez de reservar tudo para o pico.

Intuição. Pense num prédio de servidores. Sem virtualização, cada aplicação ocupa um andar inteiro (desperdício). Com virtualização, várias aplicações dividem um andar — desde

que a soma do que elas usam caiba.

2.3 Por que energia e desperdício importam

A energia **domina o custo operacional** de um data center e gera CO_2 e problemas de confiabilidade. Logo, o provedor quer ligar o *menor número de servidores* possível (e desligar os ociosos). Ao mesmo tempo, precisa *aproveitar bem* CPU e memória dos servidores ligados — senão liga poucos servidores mas usa mal cada um.

3 O problema central: alocação de VMs (*VM placement*)

Alocação de VMs é o processo de **mapear cada VM a um servidor físico**. O exemplo da Figura 1 do artigo: 7 servidores com 1 VM cada (uso de 15% a 50%) são **consolidados** em apenas 3 servidores bem ocupados (95%). Resultado: menos máquinas ligadas, menos energia.

3.1 Por que é difícil (NP-difícil)

O problema é uma variante do **empacotamento vetorial** (*vector bin packing*): empacotar “itens” (VMs) com duas dimensões (CPU e memória) em “caixas” (servidores) de capacidade limitada. Para n VMs e m servidores há m^n alocações possíveis — explosão combinatória. Por isso não dá para testar todas; usam-se **meta-heurísticas** (métodos inteligentes de busca aproximada), como a colônia de formigas.

3.2 Abordagens da literatura (citadas no artigo)

Programação linear, algoritmos genéticos, programação por restrições e heurísticas de *bin packing* (como o **First-Fit Decreasing, FFD**). A maioria otimiza *um* critério; o artigo é dos primeiros a tratar **dois objetivos** (energia e desperdício) com colônia de formigas.

4 Otimização multiobjetivo (a teoria por trás)

Quando há **vários objetivos que competem**, melhorar um pode piorar o outro. Não existe “a melhor solução”; existe um *conjunto* das melhores trocas possíveis.

4.1 Vocabulário formal

Para um problema de minimização com m variáveis de decisão e n objetivos:

$$\text{Minimizar } \vec{f}(\vec{x}) = (f_1(\vec{x}), \dots, f_n(\vec{x})), \quad \vec{x} = (x_1, \dots, x_m) \in X, \quad \vec{f} \in Y.$$

- $\vec{x}$: **vetor de decisão** (a “solução” — aqui, qual VM vai em qual servidor). X : espaço de decisão (todas as soluções possíveis).
- $\vec{f}$: **vetor objetivo** (os valores dos objetivos daquela solução — aqui, energia e desperdício). Y : espaço de objetivos.

4.2 Dominância de Pareto

Uma solução $\vec{x}_1$ **domina** $\vec{x}_2$ se as duas condições valem:

1. $\vec{x}_1$ **não é pior** que $\vec{x}_2$ em *nenhum* objetivo; e
2. $\vec{x}_1$ é **estritamente melhor** em *pelo menos um* objetivo.

Exemplo. Solução A = (energia 100, desperdício 5); B = (energia 120, desperdício 8). A domina B (melhor nos dois). Mas se C = (energia 130, desperdício 3), A *não* domina C (A gasta menos energia, porém C desperdiça menos): são **não-dominadas** entre si — ambas são compromissos válidos.

4.3 Fronteira de Pareto

O conjunto de todas as soluções **não-dominadas** forma a **fronteira de Pareto** (no espaço de objetivos) e o **conjunto de Pareto** (no espaço de decisão). É a “curva de melhores trocas”. O artigo usa este procedimento para identificar não-dominadas em um conjunto de N soluções: para cada solução i , compara com todas as outras; se alguma j a domina, marca i como dominada; ao fim, as não marcadas são as não-dominadas.

5 Colônia de Formigas (ACO)

ACO (*Ant Colony Optimization*) é uma meta-heurística inspirada no comportamento real das formigas ao procurar comida.

5.1 A metáfora biológica

Formigas exploram ao redor do ninho “aleatoriamente”. Ao andar, depositam **feromônio** (uma substância química) no caminho. Elas tendem a seguir caminhos com *mais* feromônio. Como o caminho mais curto é percorrido mais vezes no mesmo tempo, ele acumula mais feromônio e atrai mais formigas — e a colônia “descobre” coletivamente o melhor caminho. Essa comunicação indireta pelo ambiente chama-se **estigmergia**.

Tradução para o algoritmo. “Caminho” = uma sequência de decisões (aqui, colocar cada VM em um servidor). “Feromônio” = memória numérica de quais decisões apareceram em boas soluções. Boas decisões ganham feromônio e tendem a ser repetidas pelas próximas “formigas” (tentativas).

5.2 Variantes e a versão multiobjetivo

Existem o *Ant System* (AS), o **Ant Colony System** (ACS) — base do VMPACS — e o MMAS. Para vários objetivos, as variantes diferem em três pontos: (i) **quais soluções atualizam o feromônio** (as melhores de cada objetivo, ou só as não-dominadas guardadas num arquivo externo); (ii) **como o feromônio/heurística é definido** (uma matriz combinada, ou uma matriz por objetivo); (iii) **como agregar** (soma ponderada, produto ou objetivo sorteado). O VMPACS usa *uma* matriz de feromônio, *soma* as heurísticas dos dois objetivos e deixa *só as não-dominadas* reforçarem.

6 Modelagem dos recursos e dos dois objetivos

Cada servidor tem **duas dimensões** de recurso: **CPU** e **memória**. Se duas VMs (20%, 30%) e (35%, 40%) ficam no mesmo servidor, o uso vira a *soma* (55%, 70%). Impõe-se um **limite** (*threshold*) abaixo de 100% (no artigo, 90%) porque rodar no talo degrada o desempenho e a migração de VMs consome CPU.

6.1 Objetivo 2: desperdício de recursos W_j

$$W_j = \frac{|L_j^p - L_j^m| + \varepsilon}{U_j^p + U_j^m}$$

Significado de cada símbolo (para o servidor j):

- U_j^p : **CPU usada** (normalizada) — soma das CPUs das VMs nele.
- U_j^m : **memória usada** (normalizada).
- L_j^p : **CPU que sobrou** (*remaining*) $= T_j^p - U_j^p$.
- L_j^m : **memória que sobrou** $= T_j^m - U_j^m$.
- ε : número pequeno (0,0001) só para **evitar divisão por zero**.

Por que essa fórmula? O numerador $|L_j^p - L_j^m|$ mede o **desbalanceamento**: se sobra muita CPU mas pouca memória (ou vice-versa), esse servidor está “torto” e desperdiça. O denominador $U_j^p + U_j^m$ premia servidores **bem ocupados**. Logo, W_j é pequeno quando o servidor está *cheio e equilibrado*, e grande quando está *vazio ou desbalanceado*. Minimizar $\sum_j W_j$ = empacotar de forma cheia e equilibrada.

6.2 Objetivo 1: consumo de energia P_j

$$P_j = \begin{cases} (P_j^{busy} - P_j^{idle}) U_j^p + P_j^{idle}, & U_j^p > 0 \\ 0, & \text{caso contrário.} \end{cases}$$

- P_j^{idle} : potência do servidor **ocioso** (ligado, sem carga). No artigo, **162 W**.
- P_j^{busy} : potência do servidor em **uso pleno**. **215 W**.
- U_j^p : utilização de CPU (a potência cresce *linearmente* com ela — fato medido em servidores reais).

Consequência importante. Servidores ociosos são *desligados* ($P_j = 0$). Somando sobre os servidores ligados, a energia total vira $(P^{busy} - P^{idle}) \cdot (\text{CPU total}) + P^{idle} \cdot k$, onde k é o **número de servidores ligados**. Como a CPU total é fixa (todas as VMs precisam rodar), **a energia depende basicamente de quantos servidores estão ligados**. Por isso “minimizar energia” $\approx$ “usar menos servidores”.

7 Formulação matemática completa

Dados: n VMs ($i \in I$) e m servidores ($j \in J$). **Parâmetros:** R_i^p (demanda de CPU da VM i), R_i^m (demanda de memória), T_j^p e T_j^m (limites de CPU e memória do servidor), P^{idle} , P^{busy} , ε .

Variáveis de decisão:

- $x_{ij} \in \{0, 1\}$: vale **1** se a VM i vai no servidor j , senão 0.
- $y_j \in \{0, 1\}$: vale **1** se o servidor j está em uso, senão 0.

Os dois objetivos (minimizar):

$$f_1 = \sum_{j=1}^m y_j \left[(P_j^{busy} - P_j^{idle}) \sum_{i=1}^n x_{ij} R_i^p + P_j^{idle} \right] \quad (\text{energia})$$

$$f_2 = \sum_{j=1}^m y_j \frac{|(T_j^p - \sum_i x_{ij} R_i^p) - (T_j^m - \sum_i x_{ij} R_i^m)| + \varepsilon}{\sum_i x_{ij} R_i^p + \sum_i x_{ij} R_i^m} \quad (\text{desperdício})$$

Repare que $\sum_i x_{ij} R_i^p$ é exatamente U_j^p (a CPU usada no servidor j), e $T_j^p - \sum_i x_{ij} R_i^p$ é L_j^p (a CPU que sobrou). É a mesma fórmula da seção anterior, escrita com as variáveis de decisão.

Restrições:

$$\sum_{j=1}^m x_{ij} = 1, \quad \forall i \quad (2) \text{ cada VM em \textbf{exatamente um} servidor}$$

$$\sum_{i=1}^n R_i^p x_{ij} \leq T_j^p y_j, \quad \forall j \quad (3) \text{ não estourar a \textbf{CPU} do servidor}$$

$$\sum_{i=1}^n R_i^m x_{ij} \leq T_j^m y_j, \quad \forall j \quad (4) \text{ não estourar a \textbf{memória}}$$

$$x_{ij}, y_j \in \{0, 1\} \quad \forall i, j \quad (5) \text{ variáveis binárias}$$

Nas restrições (3) e (4), o y_j do lado direito garante que, se o servidor não está em uso ($y_j = 0$), nada pode ser alocado nele.

8 O algoritmo VMPACS (passo a passo)

Uma “formiga” constrói uma alocação completa. Um **movimento** é colocar a VM i no servidor j (notação *VM-Host*). A qualidade da formiga é dada pelos dois objetivos (f_1, f_2).

8.1 Feromônio τ_{ij} e seu valor inicial τ_0

τ_{ij} = “vontade” de colocar a VM i no j -ésimo servidor aberto. Começa todo igual a:

$$\tau_0 = \frac{1}{n (P'(S_0) + W(S_0))}$$

- S_0 : solução inicial dada pela heurística **FFD** (First-Fit Decreasing): ordena as VMs da maior para a menor e coloca cada uma no primeiro servidor onde cabe.
- $W(S_0)$: desperdício dessa solução inicial.
- $P'(S_0) = \sum_j P_j / P_j^{Max}$: potência *normalizada* (P_j^{Max} = potência de pico). Serve para deixar energia e desperdício na mesma escala.
- n : número de VMs.

Por quê? Define um nível de feromônio de partida coerente com a qualidade de uma solução “razoável” (a do FFD). Nem alto nem baixo demais.

8.2 Heurística η_{ij} (o “conhecimento do problema”)

Enquanto o feromônio é *aprendido*, a heurística é *calculada* e indica o quão promissor é um movimento agora. Ela soma a contribuição parcial aos dois objetivos:

$$\eta_{ij} = \eta_{ij}^{(1)} + \eta_{ij}^{(2)}, \quad \eta_{ij}^{(1)} = \frac{1}{\varepsilon + \sum_{v=1}^j P_v / P_v^{Max}}, \quad \eta_{ij}^{(2)} = \frac{1}{\varepsilon + \sum_{v=1}^j W_v}.$$

- $\eta_{ij}^{(1)}$: liga-se ao **objetivo energia**. As somas $\sum_{v=1}^j$ percorrem os servidores já abertos (1 até j). Quanto *menor* a energia parcial, *maior* fica $\eta^{(1)}$ (note o $1/(\dots)$).
- $\eta_{ij}^{(2)}$: liga-se ao **desperdício**. Mesma ideia: menos desperdício parcial $\Rightarrow$ heurística maior.

Resumo: η_{ij} é grande para movimentos que mantêm energia e desperdício baixos — exatamente o que queremos.

8.3 Como a formiga escolhe (regra pseudoaleatória proporcional)

Para escolher a próxima VM a colocar no servidor atual, sorteia-se um número $q \in [0, 1]$ e compara-se com o parâmetro q_0 :

$$i = \begin{cases} \arg \max_{u \in \Omega_k(j)} \{ \alpha \tau_{uj} + (1 - \alpha) \eta_{uj} \}, & q \leq q_0 \text{ (intensificação)} \\ s \text{ sorteado por } p_{ij}^k = \frac{\alpha \tau_{ij} + (1 - \alpha) \eta_{ij}}{\sum_{u \in \Omega_k(j)} \alpha \tau_{uj} + (1 - \alpha) \eta_{uj}}, & q > q_0 \text{ (exploração)} \end{cases}$$

- α : peso (entre 0 e 1) que equilibra **feromônio** (τ , o aprendido) *vs.* **heurística** (η , o calculado). No artigo, $\alpha = 0,45$.
- q_0 : limiar (no artigo 0,8). Com prob. q_0 a formiga **intensifica** (escolhe o melhor movimento conhecido); com prob. $1 - q_0$ ela **explora** (sorteia, para não ficar presa).
- $\Omega_k(j)$: conjunto das VMs que *ainda cabem* no servidor atual j (respeitando os limites de CPU e memória).

8.4 Atualização do feromônio (local e global)

Local (Eq. 8) — aplicada *logo após* cada movimento:

$$\tau_{ij} \leftarrow (1 - \rho_l) \tau_{ij} + \rho_l \tau_0$$

Ela *evapora* um pouco o feromônio do movimento recém-usado (puxando para τ_0), o que **evita que todas as formigas sigam o mesmo caminho** e favorece a diversidade. ρ_l é a taxa de evaporação local.

Global (Eq. 9) — aplicada ao fim de cada iteração, *só* para as soluções S do arquivo de Pareto:

$$\tau_{ij} \leftarrow (1 - \rho_g) \tau_{ij} + \frac{\rho_g \lambda}{P'(S) + W(S)}, \quad \lambda = \frac{N_A}{t - N_{Is} + 1}.$$

- ρ_g : taxa de evaporação global.
- O termo $\frac{1}{P'(S) + W(S)}$ **reforça mais** os movimentos de soluções *melhores* (menor energia + desperdício).
- λ (Eq. 10): coeficiente *adaptativo*. N_A = número de formigas; t = iteração atual; N_{Is} = iteração em que a solução S entrou no arquivo. Soluções que estão no arquivo há *mais* tempo contribuem *menos* (o λ diminui) — assim o algoritmo continua aprendendo coisas novas.

8.5 Pseudocódigo resumido

1. Inicializa o arquivo de Pareto vazio e todo $\tau_{ij} = \tau_0$.
2. Repete por M iterações:
 - (a) Cada uma das N_A formigas: abre um servidor, enche-o escolhendo VMs pela regra acima (aplicando a *atualização local* a cada movimento), abre o próximo servidor, e assim até alocar todas as VMs.
 - (b) Avalia (f_1, f_2) de cada formiga e atualiza o arquivo de não-dominadas.
 - (c) Aplica a *atualização global* usando as soluções do arquivo.
3. Devolve o arquivo de Pareto.

9 Metodologia experimental do artigo

9.1 Geração das instâncias (com correlação CPU/memória)

Para cada VM i :

$$R_i^p = \text{rand}(2\bar{R}_p); \quad R_i^m = \text{rand}(\bar{R}_m); \quad r = \text{rand}(1);$$
$$\text{se } (r < P \wedge R_i^p \geq \bar{R}_p) \vee (r \geq P \wedge R_i^p < \bar{R}_p) \Rightarrow R_i^m += \bar{R}_m.$$

- $\bar{R}_p, \bar{R}_m$: valores de referência (25% ou 45%).
- P : probabilidade que **controla a correlação** entre CPU e memória. Para $\bar{R} = 25\%$, os coeficientes de correlação ficam $-0,754, -0,348, -0,072, 0,371, 0,755$ para $P = 0; 0,25; 0,5; 0,75; 1$ (de correlação *negativa forte* a *positiva forte*).

9.2 Parâmetros

VMPACS: $N_A = 10, M = 100, \alpha = 0,45, \rho_l = \rho_g = 0,35, q_0 = 0,8$. MGGA (o genético comparado): população 12, cruzamento 0,7, mutação 0,05, 10 gerações. Instâncias com **200 VMs**, servidores = nº de VMs (pior caso), limites $T^p = T^m = 90\%$, **20 execuções** independentes (média reportada).

10 Métricas usadas no artigo

10.1 ONVG — quantas soluções a fronteira tem

$$\text{ONVG} = |Y_{\text{known}}|_c$$

É a **cardinalidade** (número) de soluções não-dominadas encontradas. **Maior é melhor** (mais opções de compromisso). Y_{known} é a fronteira calculada; $|\cdot|_c$ = contagem.

10.2 Spacing (SP) — quão uniforme é o espaçamento

$$\text{SP} = \sqrt{\frac{1}{|Y_{\text{known}}|_c - 1} \sum_i (\bar{d} - d_i)^2}, \quad d_i = \min_{j \neq i} \sum_m |f_m^i - f_m^j|.$$

- d_i : distância (de Manhattan) da solução i até a *mais próxima* da fronteira.
- $\bar{d}$: média dos d_i . SP é o *desvio-padrão* dos d_i .
- **Menor é melhor**: SP perto de 0 significa pontos *uniformemente* distribuídos.

11 Resultados do artigo

- **VMPACS vs. MGGA** (Tabela 1 do artigo): o VMPACS tem *maior* ONVG e *menor* SP em todas as correlações — fronteiras mais ricas e mais uniformes.
- **VMPACS vs. FFD vs. SACO** (Tabela 2): o FFD (heurística simples) gasta *mais* energia e desperdício (usa mais servidores); o SACO (formiga de objetivo único) fica no meio; o **VM-PACS é o melhor** nos dois objetivos, porque busca o espaço de forma global e acha soluções com menos servidores e melhor aproveitamento.
- **Escalabilidade** (Figura 4): resolve até **2000 VMs em menos de 3 minutos** — adequado a data centers grandes.

12 Conclusão do artigo

O VMPACS obtém, de forma eficiente, um conjunto de soluções não-dominadas que minimizam *ao mesmo tempo* energia e desperdício, supera um genético (MGGA) e dois métodos de objetivo único (FFD, SACO), e escala bem. É a primeira aplicação do ACS à alocação multiobjetivo de VMs.

13 Glossário de símbolos

Símbolo	Significado
n, m	número de VMs e de servidores
i, j	índice de VM e de servidor
R_i^p, R_i^m	demanda de CPU e de memória da VM i
T_j^p, T_j^m	limite (threshold) de CPU e memória do servidor j (90%)
U_j^p, U_j^m	CPU e memória <i>usadas</i> no servidor j
L_j^p, L_j^m	CPU e memória <i>restantes</i> no servidor j
x_{ij}	1 se a VM i está no servidor j (senão 0)
y_j	1 se o servidor j está em uso (senão 0)
P_j	potência (energia) do servidor j
P^{idle}, P^{busy}	potência ociosa (162 W) e em uso pleno (215 W)
W_j	desperdício do servidor j
ε	constante pequena (10^{-4}), evita divisão por 0
τ_{ij}	feromônio do movimento (VM $i \rightarrow$ servidor j)
τ_0	feromônio inicial
η_{ij}	heurística do movimento
α	peso feromônio vs. heurística (0,45)
q_0	limiar intensificação/exploração (0,8)
ρ_l, ρ_g	taxas de evaporação local e global (0,35)
N_A, M	nº de formigas (10) e de iterações (100)
λ, N_{Is}	coef. adaptativo e iteração de entrada no arquivo
$\bar{R}_p, \bar{R}_m, P$	referências e prob. de correlação (instâncias)

14 Prováveis perguntas da banca (e respostas)

- **Por que é multiobjetivo e não só “gastar menos energia”?** Porque minimizar energia (poucos servidores) pode piorar o aproveitamento (desbalanceamento). Tratar os dois juntos dá ao operador um leque de compromissos.
- **O que é a fronteira de Pareto, em uma frase?** O conjunto das soluções em que não dá para melhorar um objetivo sem piorar o outro.
- **Por que colônia de formigas e não força bruta?** Porque há m^n alocações possíveis (NP-difícil); a colônia busca boas soluções sem testar todas.
- **Para que serve o feromônio?** É a memória coletiva: movimentos que apareceram em boas soluções ficam mais prováveis de serem repetidos.
- **Qual a diferença entre τ e η ?** τ é *aprendido* ao longo das iterações; η é *calculado* na hora, com conhecimento do problema (energia e desperdício parciais).
- **Por que o limite de 90%?** Rodar perto de 100% degrada o desempenho e a migração de VMs ainda consome CPU.
- **Por que a energia “depende dos servidores”?** Porque a CPU total é fixa; somando as potências, sobra $P^{idle} \times (\text{nº de servidores})$ como termo variável.

Referências

- [1] Y. Gao, H. Guan, Z. Qi, Y. Hou, L. Liu. *A multi-objective ant colony system algorithm for virtual machine placement in cloud computing*. Journal of Computer and System Sciences, 79(8):1230–1242, 2013.
- [2] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan. *A fast and elitist multiobjective genetic algorithm: NSGA-II*. IEEE Transactions on Evolutionary Computation, 6(2):182–197, 2002.
- [3] M. Dorigo, L. M. Gambardella. *Ant colony system: a cooperative learning approach to the traveling salesman problem*. IEEE Trans. on Evolutionary Computation, 1(1):53–66, 1997.
- [4] E. Zitzler, L. Thiele. *Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach*. IEEE Trans. on Evol. Computation, 3(4):257–271, 1999.
- [5] D. A. Van Veldhuizen. *Multiobjective Evolutionary Algorithms: Classifications, Analyses, and New Innovations*. PhD thesis, Air Force Institute of Technology, 1999.